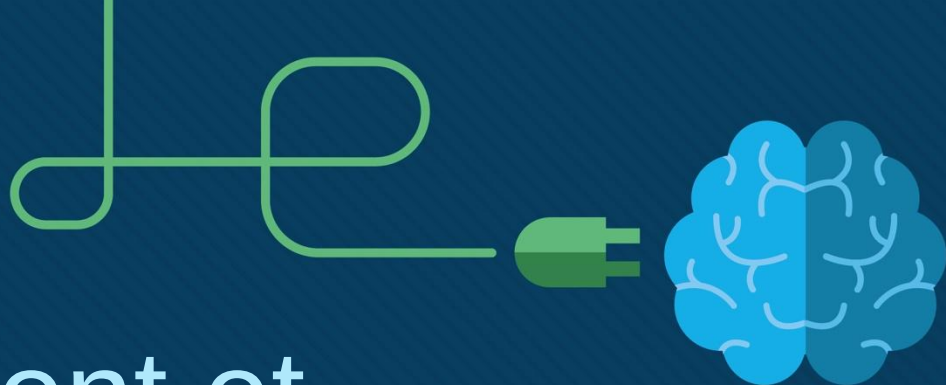




Module 3 : Développement et conception de logiciels

Contenu Pédagogique d l'instructeur

DevNet Associate v1.0



Contenu pédagogique de l'instructeur - Guide de planification du module 3

Cette présentation PowerPoint est divisée en deux parties :

- Guide de planification de l'enseignant
 - Informations pour vous aider à vous familiariser avec le module
 - Outils pédagogiques
- Présentation en classe pour le formateur
 - Diapositives facultatives que vous pouvez utiliser en classe
 - Commence à la diapositive 9

Remarque : supprimez le guide de planification de cette présentation avant de la partager.

Pour obtenir de l'aide et des ressources supplémentaires, consultez la page d'accueil de l'instructeur et les ressources du cours pour ce cours. Vous pouvez également visiter le site de développement professionnel sur www.netacad.com, la page Facebook officielle de Cisco Networking Academy ou le groupe FB Instructor Only.

À quoi s'attendre dans ce module

Pour faciliter l'apprentissage, les caractéristiques suivantes de l'interface graphique GUI peuvent être incluses dans ce module :

Fonctionnalité	Description
Travaux Pratiques	Travaux Pratiques conçus pour travailler avec des équipements physiques.
Questionnaires sur le module	Des évaluations automatiques qui intègrent les concepts et les compétences acquises tout au long de la série de rubriques présentées dans le module.
Résumé du module	Récapte brièvement le contenu du module.

Vérifiez vos connaissances

- Les activités Vérifiez votre compréhension sont conçues pour permettre aux élèves de déterminer rapidement s'ils comprennent le contenu et s'ils peuvent poursuivre ou s'ils ont besoin de revoir.
- Les exercices du module Vérifiez votre compréhension ne sont ***pas*** comptés dans la note finale des candidats.
- Il n'existe aucune diapositive distincte pour ces exercices dans le fichier PPT. Ils sont répertoriés dans les notes de la diapositive qui apparaissent avant ces exercices.

Module 3 : Activités

Quelles sont les activités associées à ce module?

N° de page	Type d'exercice	Nom de l'exercice	Facultatif ?
3.1.12	Travaux pratiques	Explorer les outils de développement Python	Recommandation
3.3.11	Travaux pratiques	Contrôle des versions logicielles avec Git	Recommandation
3.4.6	Travaux pratiques	Explorer les classes Python	Recommandation
3.5.7	Travaux pratiques	Créer un test unitaire Python	Recommandation
3.6.6	Travaux pratiques	Analyser différents types de données avec Python	Recommandation

Module 3 : Meilleures pratiques

Avant d'enseigner le contenu du module 3, l'enseignant doit:

- Passez en revue les activités et les évaluations de ce module.
- Essayez d'inclure autant de questions que possible pour maintenir l'intérêt des élèves pendant la présentation en classe.

Rubrique 3.1

- Demandez aux apprenants ce qu'ils comprennent du terme Développement logiciel.
- Spécifiez si les apprenants ont entendu parler du cycle de vie du développement logiciel. Demandez-leur de partager leur compréhension avec la classe.
- Discutez des différentes méthodologies de développement de logiciels. Divisez la classe en groupes et discutez de quelle méthodologie est la meilleure dans le scénario de développement logiciel d'aujourd'hui.

Module 3 : Meilleures pratiques (suite)

Rubrique 3.2

- Demandez aux apprenants de se référer au livre « Design Patterns - Elements of Reusable Object-Oriented Software » du Gang of Four (GoF) pour en savoir plus sur Software Design Patterns.
- Fournissez aux apprenants plus d'informations sur les deux modèles de conception les plus couramment utilisés : le modèle de conception Observer (un modèle de conception comportementale) et le modèle de visualisation MVC (Model-View-Controller).

Rubrique 3.3

- Sondez si les apprenants ont entendu parler des systèmes de contrôle de version. Discutez le sujet en détail.
- Discutez Git et GitHub.

Rubrique 3.4

- Demandez aux apprenants de partager leur compréhension des méthodes, des fonctions, des modules et des classes.
- Demandez aux apprenants ce qu'ils comprennent du terme « Clean Code » ? Invite les apprenants à discuter et à trouver des raisons pour lesquelles les développeurs devraient écrire du code propre.

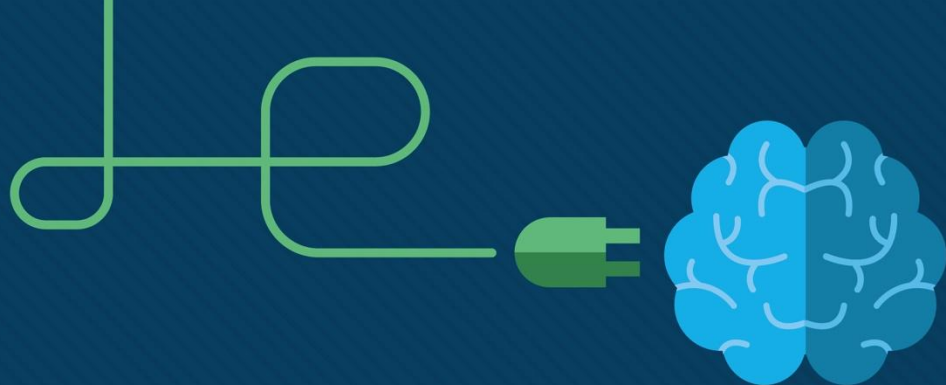
Module 3 : Meilleures pratiques (suite)

Rubrique 3.5

- Discutez l'importance de revoir le code.
- Discutez en détail des différents types de tests.
- Expliquez ce qu'est le développement axé sur les tests

Rubrique 3.6

- Discuter le formatage des données.
- Demandez aux apprenants s'ils ont entendu parler de XML, JSON et YAML. Demandez-leur de partager leurs connaissances sur le sujet.



Module 3 : Développement et conception de logiciels

DevNet Associate v1.0



Objectifs du module

Titre du module : Développement et conception de logiciels

Objectif du module: Utiliser les meilleures pratiques en matière de développement et de conception de logiciels.

Titre du Rubrique	Objectif du Rubrique
Développement de logiciels	Comparer les méthodologies de développement de logiciels.
Modèles de conception de logiciels	Décrire les bénéfices des divers modèles de conception de logiciels.
Systèmes de contrôle des versions	Implémenter le contrôle de la version logicielle à l'aide de GIT.
Les bases du codage	Expliquer les bonnes pratiques en matière de codage.
Examen et essai du code	Utiliser le test unitaire Python pour évaluer le code.
Comprendre les formats de données	Utiliser Python pour analyser les différents formats de données et de messages.

3.1 Développement de logiciels

Introduction

- Le processus de développement logiciel est également connu sous le nom de cycle de vie du développement logiciel (SDLC).
- SDLC est plus qu'un simple codage et inclut également des exigences de collecte, de création d'une preuve de concept, de test et de correction de bugs.

Cycle de vie du développement de logiciels (SDLC)

- SDLC est le processus de développement de logiciels, à partir d'une idée et se terminant par la livraison. Ce processus se compose de six phases. Chaque phase est prise en compte à partir des résultats de la phase précédente.
- SDLC est le processus de développement de logiciels, à partir d'une idée et se terminant par la livraison. Ce processus se compose de six phases. Chaque phase est prise en compte à partir des résultats de la phase précédente.
- Bien que les méthodes de cascade soient encore largement utilisées aujourd'hui, elles sont progressivement remplacées par des méthodes plus adaptatives et flexibles qui produisent un meilleur logiciel, plus rapide et moins douloureux. Ces méthodes sont collectivement appelées « développement agile ».



Phase des exigences et de l'analyse

- La phase d'analyse des besoins consiste à explorer la situation actuelle des intervenants, leurs besoins et contraintes, l'infrastructure actuelle, etc. et à déterminer le problème à résoudre par le logiciel.
- Après avoir recueilli les exigences, l'équipe analyse les résultats pour déterminer ce qui suit :
 - Est-il possible de développer le logiciel en fonction de ces exigences, et peut-il être fait dans les limites du budget ?
 - Le calendrier de développement comporte-t-il des risques et, si oui, lesquels ?
 - Comment le logiciel sera-t-il testé ?
 - Quand et comment le logiciel sera-t-il livré ?
- À l'issue de cette phase, la méthode classique de la cascade suggère de créer un document de spécification des exigences logicielles (SRS), qui énonce les exigences et la portée du logiciel, et le confirme méticuleusement auprès des intervenants (stakeholders).

Phases de conception et de mise en œuvre

Conception

- Au cours de la phase de conception, les architectes et les développeurs de logiciels conçoivent le logiciel sur la base du SRS fourni.
- À la fin de la phase, l'équipe crée des documents de conception de haut niveau (HLD) et de conception de bas niveau (LLD).

Mise en œuvre

- La phase de mise en œuvre est également appelée phase de codage ou de développement.
- Comme tous les composants et modules sont construits au cours de cette phase, il s'agit de la phase la plus longue du cycle de vie.
- À la fin de la phase, le code fonctionnel qui met en œuvre toutes les exigences du client est prêt à être testé.

Phases de test, de déploiement et de maintenance

Test

- Dans cette phase, le code est installé dans l'environnement de test
- Des tests fonctionnels, des tests d'intégration, des tests de performance et des tests de sécurité sont effectués.
- Les tests se poursuivent jusqu'à ce que tous les codes soient exempts de bug et réussissent tous les tests. A la fin de cette phase, un logiciel de haute qualité, sans bug, est prêt pour la production.

Déploiement

- Au cours de cette phase, le logiciel est installé dans l'environnement de production.
- À la fin de la phase, le responsable produit distribue le logiciel final aux utilisateurs finaux.

Maintenance

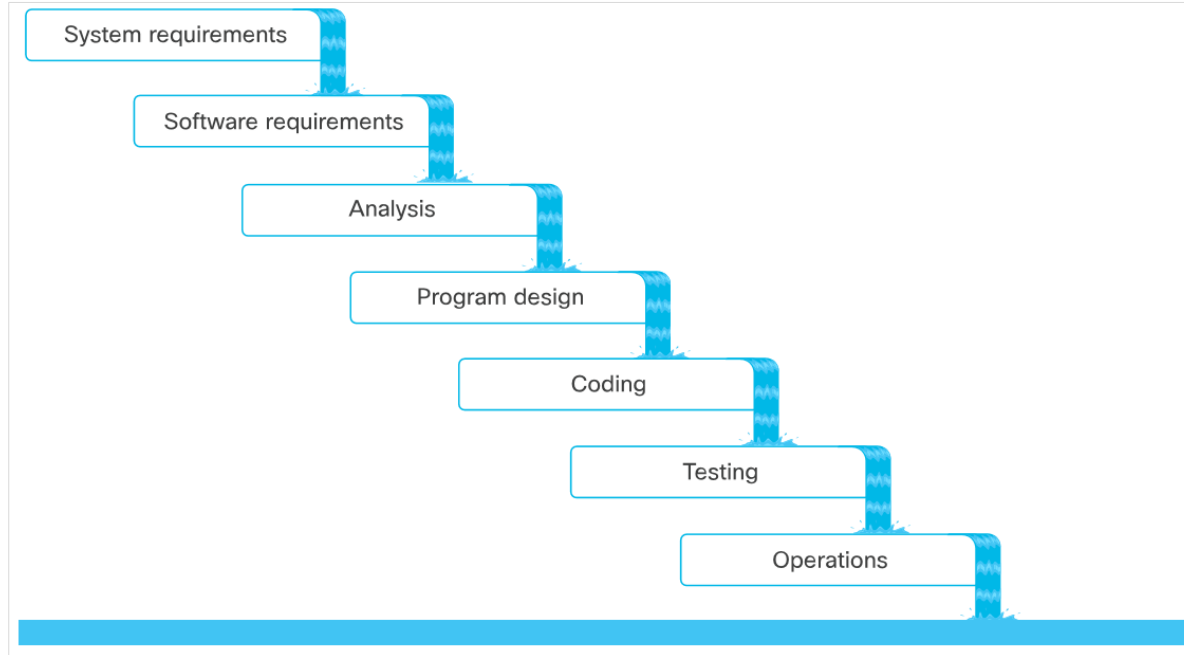
- Au cours de la phase de maintenance, l'équipe :
 - Fournit un soutien aux clients
 - Correction des bogues trouvés dans la production
 - Fonctionne sur les améliorations logicielles
 - Rassemble les nouvelles demandes du client
- À la fin, l'équipe travaille sur la prochaine itération et version du logiciel.

Méthodologies de développement de logiciels

- Une méthodologie de développement logiciel est également connue sous le nom de modèle de cycle de vie du développement logiciel.
- Les trois méthodologies les plus populaires sont :
 - Waterfall
 - Agile.
 - Lean
- Le type de méthodologie à utiliser dépend des éléments suivants :
 - Type de projet
 - Durée du projet
 - Taille de l'équipe.

Développement de logiciel Waterfall

- Le modèle original de Waterfall a été créé par Winston W. Royce.
- Son modèle original comprenait sept phases :



Développement de logiciel Agile

- La méthode agile est flexible et axée sur le client.
- Un groupe de 17 développeurs de logiciels a présenté le Manifeste pour le développement logiciel agile, également connu sous le nom de Manifeste Agile, en 2001. Selon le Manifeste Agile, les valeurs d'Agile sont :
 - Individus et interactions sur les processus et les outils
 - Un logiciel de travail plutôt qu'une documentation complète
 - Collaboration des clients lors de la négociation des contrats
 - Répondre au changement au lieu de suivre un plan
- Le manifeste Agile énumère 12 principes différents :

Principes du Manifeste Agile			
Priorité au client	Collaboration	Logiciels de travail	Simplicité
Adopter le changement et s'adapter	Équipes motivées	Travailler à un rythme durable	Des équipes auto-organisées
Livraison fréquente de logiciels de travail	Conversations en face à face	environnement Agile	Amélioration continue

Méthodes agiles

- Les méthodes Agile populaires sont :
 - **Agile Scrum** : Scrum se concentre sur de petites équipes auto-organisatrices qui se rencontrent quotidiennement pendant de courtes périodes et travaillent dans des sprints itératifs.
 - **Lean** : La méthode Lean met l'accent sur l'élimination de l'effort gaspillé dans la planification et l'exécution, et la réduction de la charge cognitive du programmeur.
 - **Programmation Extreme (XP)** : XP aborde délibérément les types spécifiques de problèmes de qualité de vie rencontrés par les équipes de développement de logiciels.
 - **Développement axé sur les fonctionnalités (Feature-Driven Development) (FDD)** : La FDD prescrit que le développement du logiciel doit se faire en fonction d'un modèle global, décomposé, planifié, conçu et construit fonction par fonction.

Méthodes agiles (suite)

- **Des sprints**

- Un sprint est une période de temps spécifique, généralement entre 2 et 4 semaines, au cours de laquelle chaque équipe assume autant de tâches (aussi appelées histoires d'utilisateurs) qu'elle estime pouvoir accomplir. Lorsque le sprint est terminé, le logiciel doit être opérationnel et livrable.
- La durée du sprint est déterminée avant le début du processus et devrait rarement changer.

- **Backlog**

- Le carnet de commandes (Backlog) se compose de toutes les fonctionnalités du logiciel, dans une liste hiérarchisée.

- **Histoires d'utilisateurs**

- Une histoire d'utilisateur est une déclaration simple de ce dont un utilisateur (ou un rôle) a besoin, et pourquoi. Chaque histoire d'utilisateur doit être assez petite pour qu'une seule équipe puisse le terminer en un seul sprint.
- Le modèle suggéré pour une histoire d'utilisateur est le suivant :

En tant que **<utilisateur|role >**, je voudrais **<action>**, afin que **<valeur|avantage>**

Méthodes agiles (suite)

Équipes Scrum

- Les équipes Scrum sont interfonctionnelles, collaboratives, autogérées et autonomes.
- Les équipes de mêlée (scrum) ne devraient pas être plus grandes que 10 personnes.
- Le maître de mêlée devrait avoir une réunion de stand-up quotidienne avec l'équipe à une heure fixe tous les jours pendant pas plus de 15 minutes.
- L'objectif est de passer en avant des tâches importantes qui ont été terminées, sont en cours ou sont sur le point d'être lancées.

Développement de logiciels Lean

- Le développement de logiciels Lean est basé sur les principes de Lean Manufacturing, qui visent à minimiser le gaspillage et à maximiser la valeur pour le client.
- Les sept principes du lean, donnés dans le livre « Lean Software Development : An Agile Toolkit », sont les suivants :
 - Éliminer les déchets (Eliminate waste)
 - Amplifier l'apprentissage
 - Décider le plus tard possible
 - Livrer aussi vite que possible
 - Renforcer l'équipe (Empower the team)
 - Renforcer l'intégrité
 - Optimiser l'ensemble (Optimize the whole)

Développement de logiciels Lean (suite)

Éliminer les déchets (Eliminate waste)

- C'est le principe de l'allégement le plus fondamental.
- Il y a sept déchets de développement de logiciels :
 - Travail partiellement fait
 - Processus supplémentaires
 - Fonctionnalités supplémentaires
 - Changement de tâche
 - Attente
 - Motion
 - Défauts

Développement de logiciels Lean (suite)

Amplifier l'apprentissage avec des sprints courts

- Pour pouvoir régler un logiciel avec précision, il doit y avoir de fréquentes et courtes itérations du logiciel en fonctionnement. Cela permet les éléments suivants :
 - Les développeurs apprennent plus rapidement
 - Les clients peuvent donner leurs commentaires plus tôt
 - Les fonctionnalités peuvent être ajustées afin qu'elles apportent plus de valeur aux clients

Décider le plus tard possible

- En cas d'incertitude, il est préférable de retarder le processus décisionnel jusqu'au plus tard possible. En effet, il est préférable de fonder les décisions sur des faits plutôt que sur des opinions ou des spéculations.

Livrer aussi vite que possible

Livrer aussi vite que possible	
Permet aux clients de fournir un retour d'information	Ne permet pas aux clients de changer d'avis
Permet aux développeurs d'amplifier l'apprentissage	Permet à chacun de prendre des décisions plus rapidement
Fournit aux clients les fonctionnalités requises	Produit moins de déchets

Développement de logiciels Lean (suite)

Renforcer l'équipe (Empower the team)

- Chaque personne doit être autorisée à prendre des décisions dans le domaine de sa propre expertise.

Renforcer l'intégrité

- L'intégrité du logiciel est lorsque le logiciel répond aux besoins du client tout en conservant l'utilité pour le client.

Optimiser l'ensemble (Optimize the whole)

- Le logiciel doit être construit de manière cohérente. La valeur du logiciel en souffrira si chaque expert se concentre uniquement sur son expertise et ne tient pas compte des ramifications de ses décisions sur le reste du logiciel.

Travaux pratiques - Explorez les outils de développement Python

Au cours de ces travaux pratiques, vous aborderez les points suivants :

- **Partie 1:** Lancer la DEVASC VM
- **Partie 2 :** Révision de l'installation de Python
- **Partie 3 :** Environnements virtuels PIP et Python
- **Partie 4 :** Partager votre environnement virtuel

3.2 Les modèles de conception de logiciels

Introduction

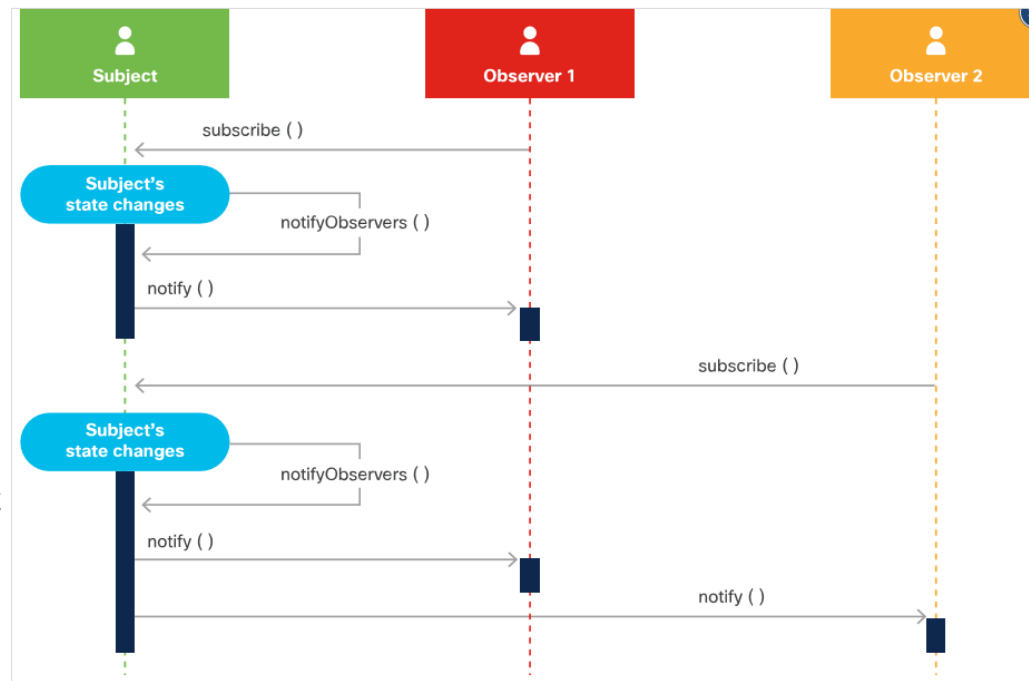
- Les modèles de conception de logiciels sont des solutions de meilleure pratique pour résoudre les problèmes courants dans le développement de logiciels.
- Les modèles de conception sont indépendants de la langue.
- En 1994, Erich Gamma, Richard Helm, Ralph Johnson et John Vlissides (connu sous le nom de Gang of Four (GoF)) ont publié un livre intitulé Design Patterns - Elements of Reusable Object-Oriented Software. Les modèles identifiés sont les suivants :
 - Programme vers une interface, pas une implémentation.
 - Favoriser la composition d'objet par rapport à l'héritage de classe.
- Les modèles de conception de logiciels ont déjà été prouvés pour leur succès, de sorte que leur utilisation peut accélérer le développement.

Les modèles de conception originaux

- Le Gang des Quatre a divisé les modèles en trois catégories principales :
 - Créatif (Creational)
 - Structurelle (Structural)
 - Comportemental (Behavioral)
- Ils ont énuméré 23 modèles de design.
- Deux des modèles de conception les plus couramment utilisés sont :
 - Le modèle de conception Observer (un modèle de conception comportementale)
 - Le contrôleur de vue du modèle (MVC)

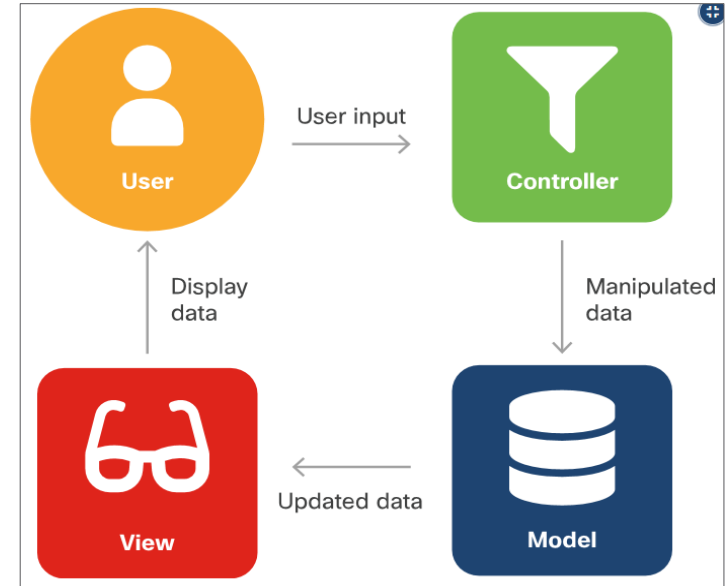
Modèle de conception d'observateur

- Le modèle de conception d'observateur est une conception de notification d'abonnement qui permet aux objets de recevoir des événements lorsqu'il y a des modifications apportées à un objet qu'ils observent.
- Pour implémenter ce mécanisme d'abonnement :
 - Le sujet doit avoir la capacité de stocker une liste de tous ses observateurs.
 - Le sujet doit avoir des méthodes pour ajouter et retirer des observateurs.
- L'avantage du modèle de conception d'observateur est que les observateurs peuvent obtenir des données en temps réel du sujet lorsqu'un changement se produit.
- Les mécanismes d'abonnement offrent toujours de meilleures performances que d'autres options, telles que l'interrogation.



Model-View-Controller (MVC)

- Le modèle de conception Model-View-Controller (MVC) vise à simplifier le développement d'applications qui dépendent des interfaces utilisateur graphiques.
- MVC résume le code et la responsabilité en trois composantes différentes :
 - **Modèle** : Le modèle est la structure de données de l'application et est responsable de la gestion des données, de la logique et des règles de l'application. Il obtient l'entrée du contrôleur.
 - **Vue** : La vue est la représentation visuelle des données.
 - **Contrôleur** : Le contrôleur est l'intermédiaire entre le modèle et la vue. Il prend l'entrée de l'utilisateur et la manipule pour s'adapter au format du modèle ou de la vue.
- L'avantage de MVC est que chaque composant peut être construit en parallèle.



3.3 Systèmes de contrôle des versions

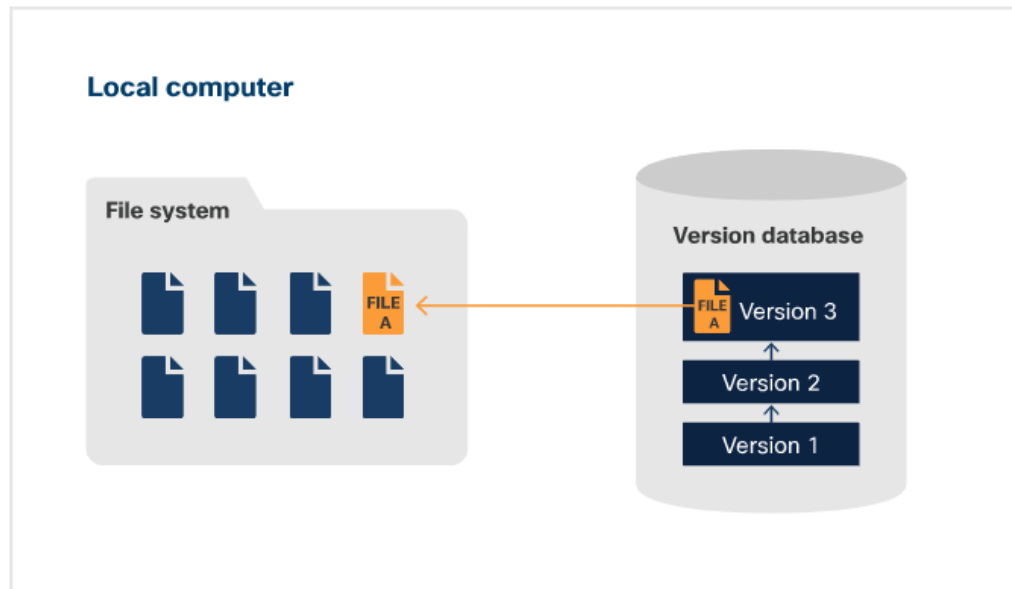
Types de systèmes de contrôle de version

- Le contrôle de version, également appelé systèmes de contrôle de version, contrôle de révision ou contrôle de source, est un moyen de gérer les modifications apportées à un ensemble de fichiers afin de conserver un historique de ces modifications.
- Les avantages du contrôle de version sont les suivants :
 - Permet la collaboration
 - Responsabilité et visibilité
 - Travail isolé
 - Sécurité
 - Travaillez où que vous soyez
- Il existe trois types de systèmes de contrôle de version :
 - Local
 - Centralisé
 - Distribué

Types de systèmes de contrôle de version (suite)

Système de contrôle de version local(LVCS)

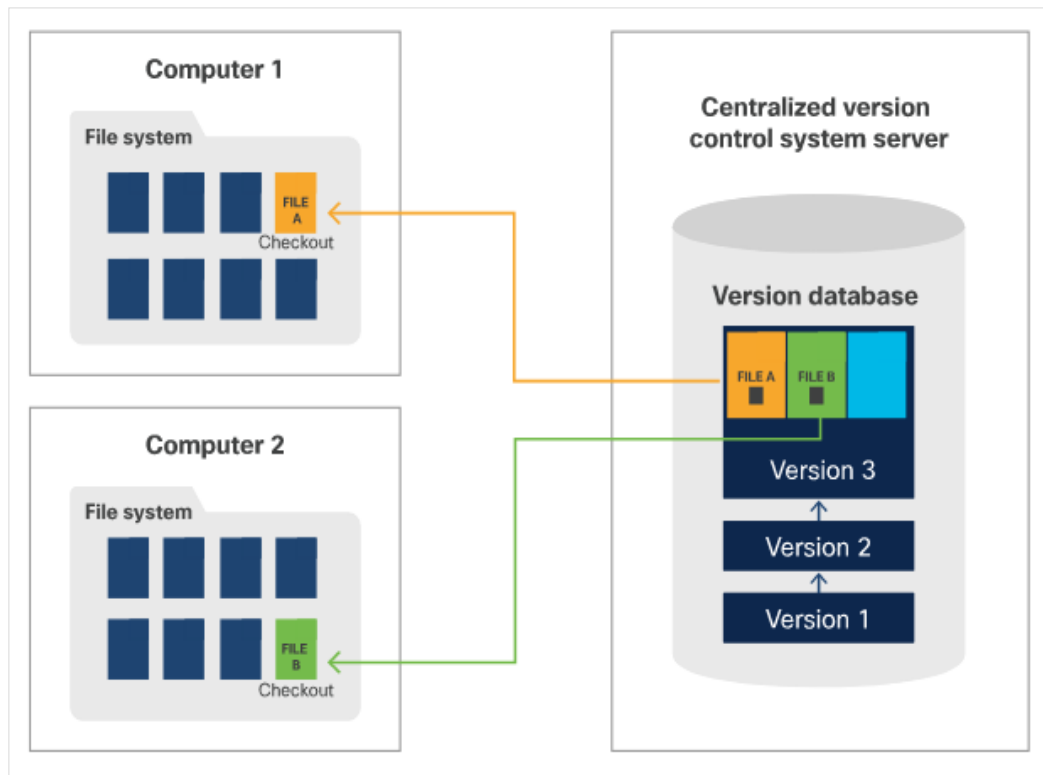
- LVCS utilise une base de données simple pour suivre toutes les modifications apportées au fichier.
- Dans la plupart des cas, le système stocke le delta entre les deux versions du fichier.
- Lorsque l'utilisateur veut revenir au fichier, le delta est inversé pour accéder à la version demandée.



Types de systèmes de contrôle de version (suite)

Système de contrôle de version centralisé (CVCS)

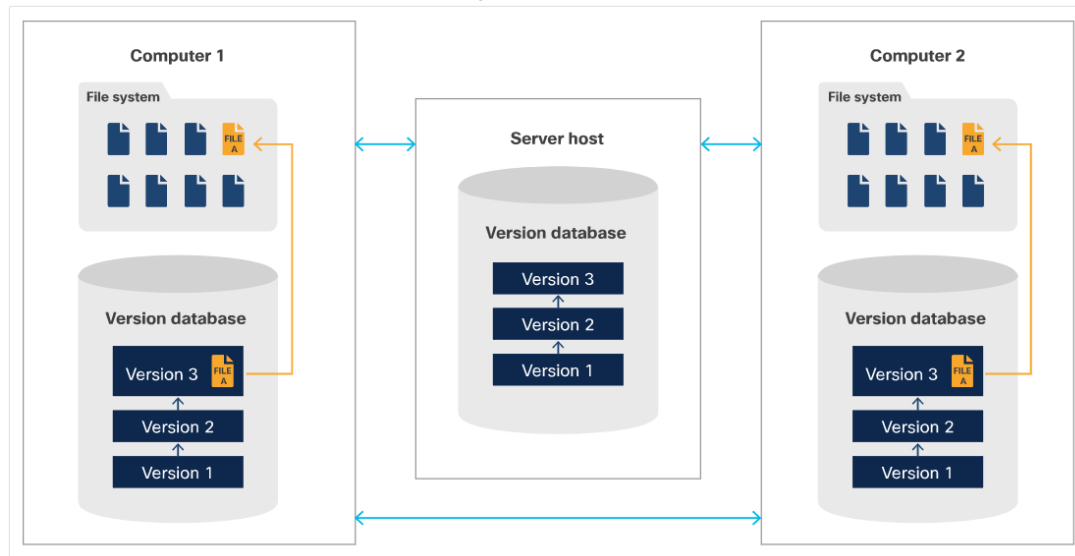
- CVCS utilise un modèle serveur-client.
- Le référentiel est stocké dans un emplacement centralisé, sur un serveur.
- Dans CVCS, une seule personne peut travailler sur un fichier particulier à la fois.
- Une personne doit récupérer le fichier pour le verrouiller et effectuer les modifications requises et l'enregistrer une fois terminé.



Types de systèmes de contrôle de version (suite)

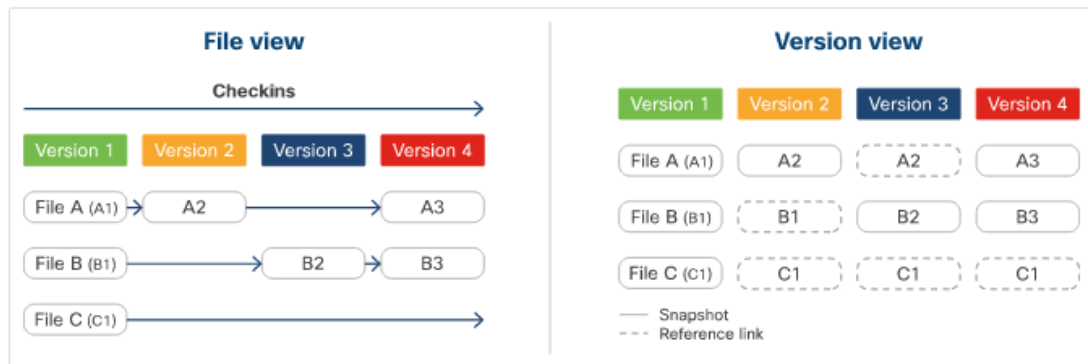
Système de contrôle de version distribué(DVCS)

- DVCS est un modèle peer-to-peer.
- Le référentiel peut être stocké sur un système client, mais il est généralement stocké dans un service d'hébergement de référentiel.
- Dans DVCS, chaque individu peut travailler sur n'importe quel fichier, en même temps, car le fichier local dans la copie de travail est en cours de modification. Par conséquent, le verrouillage n'est pas nécessaire.
- Lorsque la personne a apporté les modifications, elle pousse le fichier vers le référentiel principal qui se trouve dans le service d'hébergement du référentiel, et le système de contrôle de version détecte tout conflit entre les modifications de fichiers.



Git

- Git est une implémentation open source d'un système de contrôle de version distribué qui est actuellement la dernière tendance dans le développement de logiciels.
- Un client Git doit être installé sur une machine cliente. Il est disponible pour macOS, Windows et Linux/Unix.
- Une différence clé entre Git et les autres systèmes de contrôle de version est que Git stocke les données sous forme d'instantanés au lieu de différences (le delta entre le fichier actuel et la version précédente).

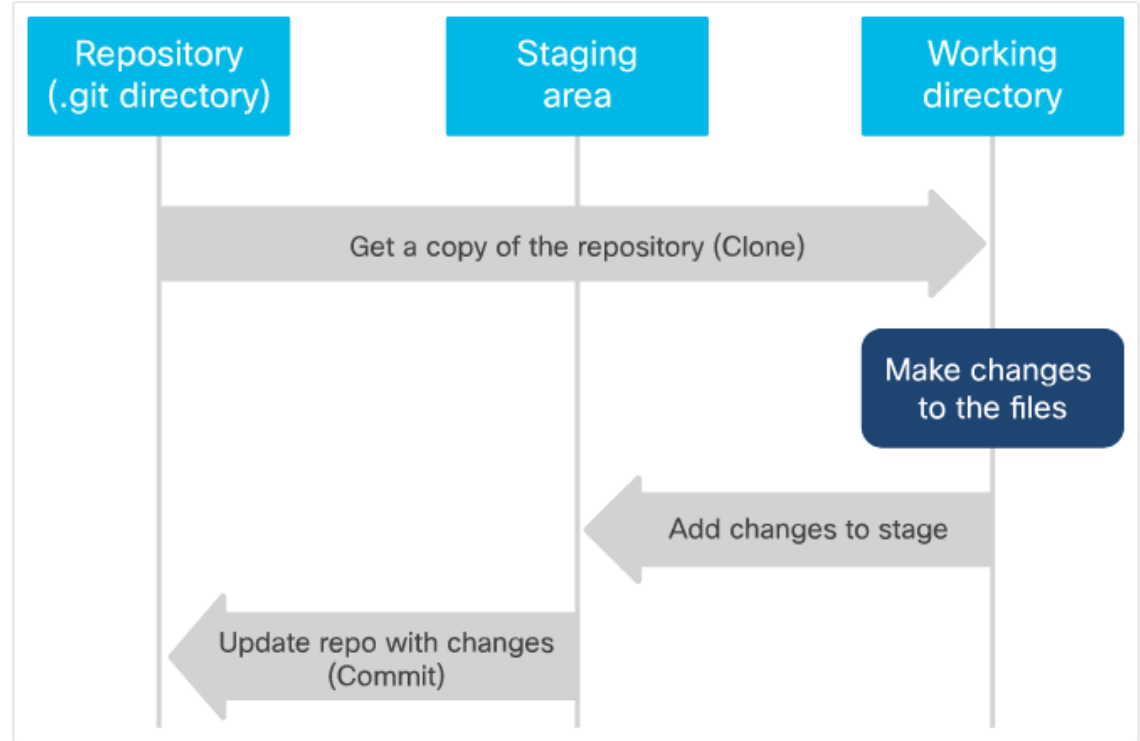


- Si le fichier ne change pas, Git utilise un lien de référence vers le dernier instantané du système au lieu de prendre un instantané nouveau et identique.

Systèmes de contrôle de version

Git (suite)

- Git est organisé par 3s- trois étapes et trois états.
- Les trois étapes sont les suivantes :
 - Référentiel (le répertoire .git)
 - Répertoire de travail
 - Zone de transit
- Les trois États sont les suivants :
 - Impliqués
 - Modifié
 - Mis en scène

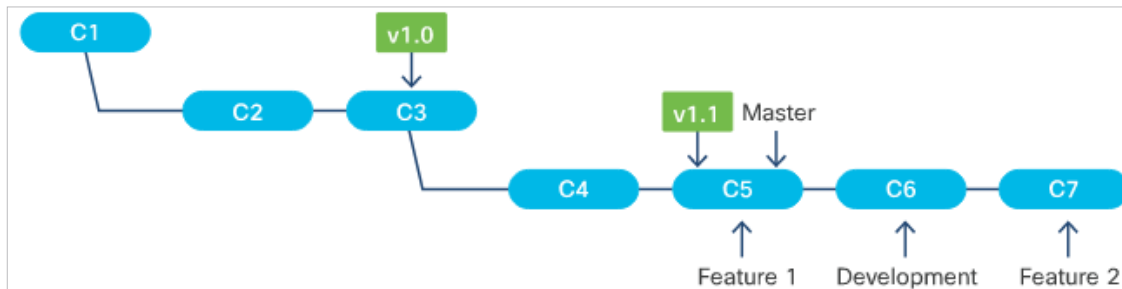
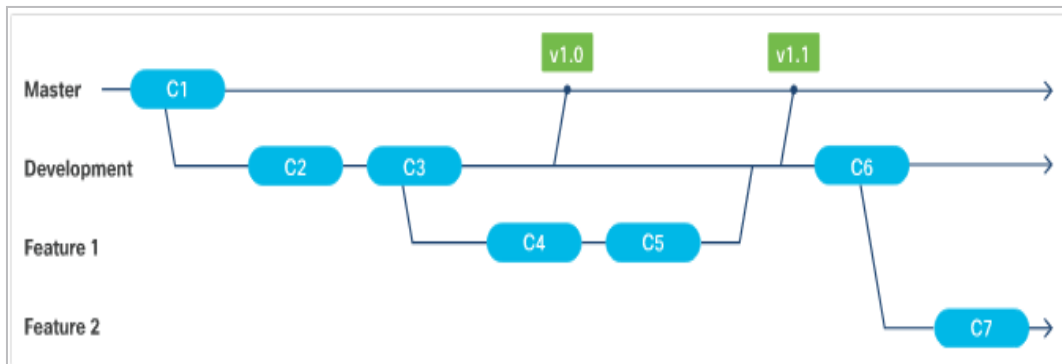


Référentiels locaux contre référentiels distants

- Git a deux types de référentiels : local et distant.
- Un référentiel local est stocké sur le système de fichiers d'une machine cliente, qui est le même sur lequel les commandes git sont exécutées.
- Un référentiel distant est stocké ailleurs que l'ordinateur client, généralement un serveur ou un service d'hébergement de référentiel.
- Un référentiel distant avec Git continue d'être un DVCS car le référentiel distant contiendra le référentiel complet, qui inclut le code et l'historique des fichiers.
- Lorsqu'une machine cliente clone le référentiel, elle obtient le référentiel complet sans avoir à le verrouiller, comme dans un CVCS.
- Une fois le référentiel local cloné à partir du référentiel distant ou le référentiel distant créé à partir du référentiel local, les deux référentiels sont indépendants l'un de l'autre jusqu'à ce que les modifications de contenu soient appliquées à l'autre branche par l'exécution manuelle d'une commande Git.

Qu'est-ce que le branchement (Branching) ?

- Le branchement permet aux utilisateurs de travailler sur le code indépendamment sans affecter le code principal dans le référentiel. Lorsqu'un référentiel est créé, le code est automatiquement placé sur une branche appelée Master.
- Les branches peuvent être locales ou distantes, et elles peuvent être supprimées et avoir leur propre historique, zone de transit et répertoire de travail.
- La création de branche de Git est légère, et le basculement entre les branches est presque instantané.
- Lorsqu'un utilisateur passe d'une branche à une autre, le code dans son répertoire de travail et les fichiers de la zone de transit changent en conséquence, mais les répertoires du dépôt (.git) restent **inchangés**.



GitHub et autres fournisseurs

- Git et GitHub ne sont pas les mêmes.
- Alors que Git est une implémentation du contrôle de version distribué et fournit une interface de ligne de commande, GitHub est un service fourni par Microsoft qui implémente un service d'hébergement de référentiel avec Git.
- En plus de fournir la fonctionnalité de contrôle de version distribuée et de gestion du code source de Git, GitHub fournit des fonctionnalités supplémentaires telles que :
 - révision du code
 - documentation
 - gestion de projets
 - suivi des bugs
 - demande de fonction
- GitHub a introduit le concept de « demande d'extraction », qui est un moyen de formaliser une demande par un contributeur pour examiner les changements tels que le nouveau code, les modifications apportées au code existant, etc., dans la branche du contributeur pour inclusion dans les branches principales du projet ou d'autres branches programmées.

Commandes Git

Configuration de Git

- Pour configurer Git, utilisez l'option `—global` pour définir les paramètres globaux initiaux.

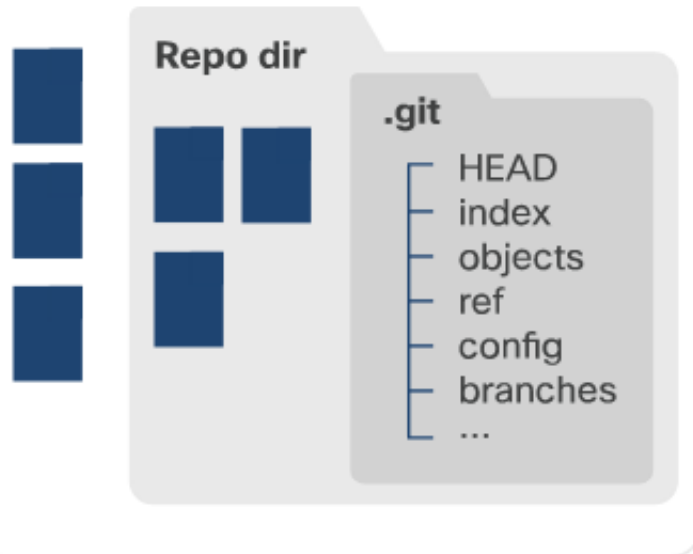
Command: `git config --global key value`

Créer un nouveau dépôt Git

- Git fournit une commande `git init` pour créer un dépôt Git vide, ou faire d'un dossier existant un dépôt Git.
- Lorsqu'un projet nouveau ou existant devient un dépôt Git, un répertoire `.git` masqué est créé dans ce dossier de projet.
- Le répertoire `.git` est le référentiel qui contient les métadonnées telles que les fichiers compressés, l'historique de validation et la zone de transit. De plus, Git crée également la branche maître.

Local computer

File system



Systèmes de contrôle de version

Commandes Git (suite)

Commande: `git init`

- Pour faire d'un projet nouveau ou existant un dépôt Git, utilisez la commande suivante :

```
$ git init <project directory>
```

où `<project directory>` est le chemin absolu ou relatif vers le projet nouveau ou existant.

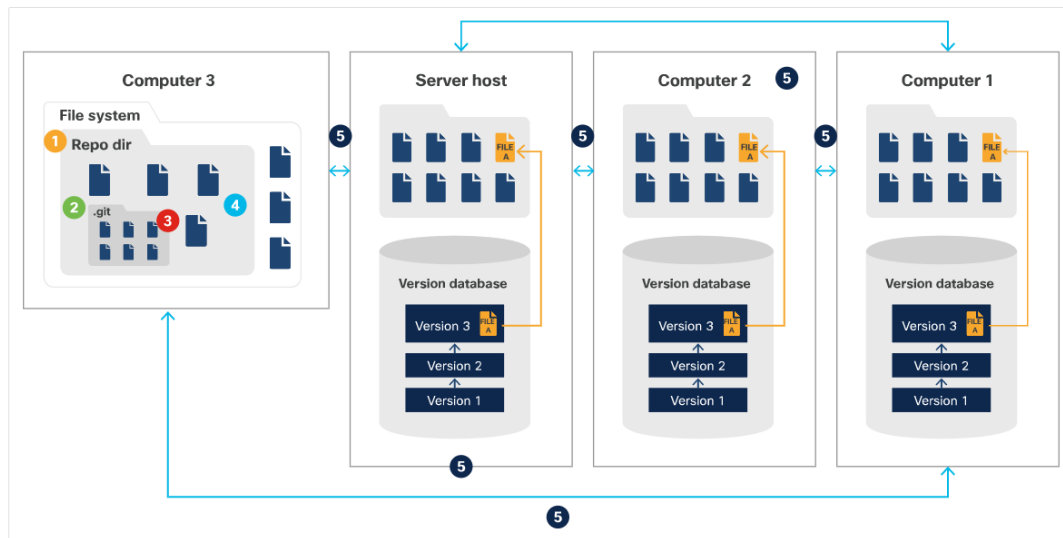
- Pour un nouveau dépôt Git, le répertoire dans le chemin fourni sera créé en premier, suivi de la création du répertoire `.git`.

Obtenir un référentiel Git existant

- Commande: `git clone <repository> [target directory]`

où `<repository>` est l'emplacement du référentiel à cloner.

- Git prend en charge quatre protocoles de transport principaux pour accéder à `<repository>`: Local, Secure Shell (SSH), Git et HTTP.



Commandes Git (suite)

Afficher les fichiers modifiés dans le répertoire de travail

- Git fournit une commande `git status` pour obtenir une liste de fichiers qui ont des différences entre le répertoire de travail et la branche parent.
- **Command:** `git status`

Comparer les modifications entre les fichiers

- Git fournit une commande `git diff` qui est essentiellement un outil générique de comparaison de fichiers.
- **Commande :** `git diff`
 - Lorsque vous utilisez la commande `git diff`, le fichier n'a pas besoin d'être un fichier suivi `Git`.

Ajout et suppression de fichiers

Ajout de fichiers à la zone intermédiaire

- **Commande :** `git add`
- Cette commande peut être utilisée plusieurs fois avant la mise à jour du dépôt Git (en utilisant `commit`).
- Seuls les fichiers spécifiés dans la commande `git` peuvent être ajoutés à la zone de transit
- Pour ajouter un seul fichier à la zone de transit:

```
$ git add <file path>
```

- Pour ajouter plusieurs fichiers à la zone de transit, où `<file path>` est le chemin absolu ou relatif du fichier à ajouter à la zone de transit.

```
$ git add <file path 1> ... <file path n>
```

- Pour ajouter tous les fichiers modifiés à la zone de transit : `$ git add`.



Ajout et suppression de fichiers (suite)

Suppression de fichiers du référentiel Git

- Il existe deux façons de supprimer des fichiers du dépôt Git.
- **Option 1** : la commande `git rm` est utilisée pour supprimer des fichiers du dépôt Git et les ajouter à la zone de transit.
- **Commande** : `git rm`
- Pour supprimer le ou les fichiers spécifiés du répertoire de travail et ajouter la modification à la zone de transit, utilisez la commande suivante :
`$ git rm <file path 1> ... <file path n>`

où `<file path>` est le chemin absolu ou relatif du fichier à supprimer du dépôt Git.



Ajout et suppression de fichiers (suite)

- Pour ajouter le(s) fichier(s) spécifié(s) à supprimer de la zone de transit sans supprimer le(s) fichier(s) lui-même(s) du répertoire de travail, utilisez la commande suivante :

```
$ git rm --cached <file path 1> ... <file path n>
```

La commande git rm ne fonctionnera pas si le fichier est déjà dans la zone de transit avec des modifications.

- **Option 2** : Cette option est un processus en deux étapes. Utilisez d'abord la commande standard du système de fichiers pour supprimer le ou les fichiers, puis ajoutez le fichier à la scène à l'aide de la commande Git.

```
$ rm <gile path 1>...  
<gile path n>$ git add <gile path 1>... <file path n>
```

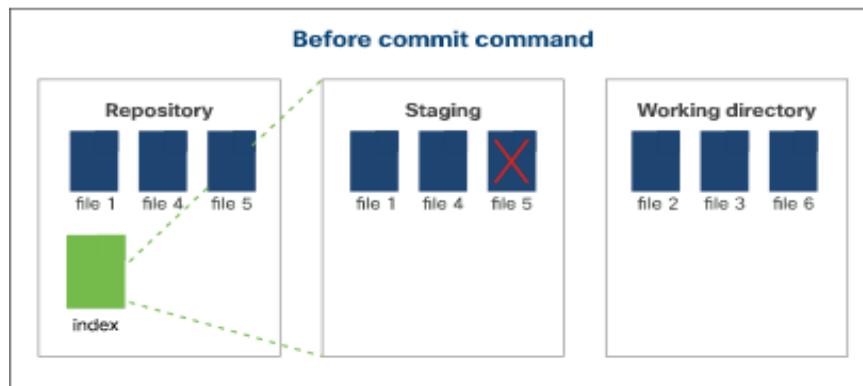
Ce processus en deux étapes équivaut à l'utilisation du `git rm <gile path 1>... <gile path n>` commande. L'utilisation de cette option ne permet pas de conserver le fichier dans le répertoire de travail.

Mise à jour des référentiels

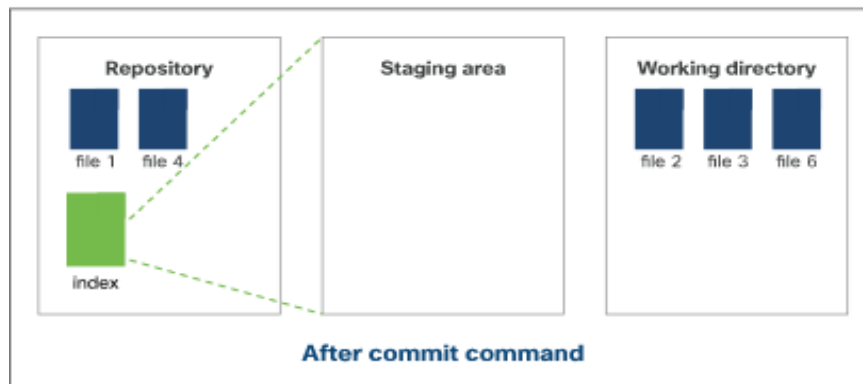
Mise à jour du référentiel local avec les modifications apportées à la zone de transit

Command: git commit

- Cette commande combine toutes les modifications de contenu dans la zone de transit en un seul commit et met à jour le dépôt Git local.
- Pour valider les modifications à partir de la zone de transit, utilisez la commande suivante :
\$ git commit
- Pour valider les modifications de la zone de transit avec un message, utilisez la commande suivante :
\$ git commit -m "<message>"



```
git commit -m "Adding new files"
```



Mise à jour des référentiels (suite)

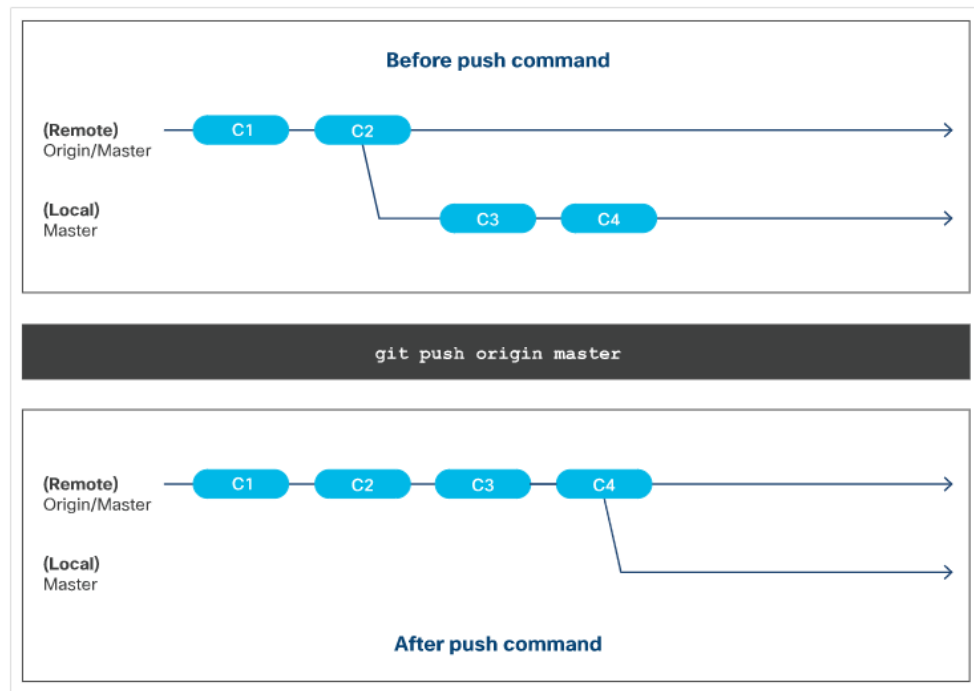
Mise à jour du référentiel distant

Commande : git push

- Cette commande ne s'exécutera pas correctement s'il y a un conflit avec l'ajout des modifications du dépôt Git local au dépôt Git distant.
- Pour mettre à jour le contenu du référentiel local vers une branche particulière du référentiel distant, utilisez la commande suivante :

```
$ git push origine <branch name>
```

- Pour mettre à jour le contenu du référentiel local vers la branche maître du référentiel distant, utilisez la commande : **\$ git push origin master**



Mise à jour des référentiels (suite)

Mise à jour de votre copie locale du référentiel

- Les copies locales du dépôt Git ne sont pas automatiquement mises à jour lorsqu'un autre contributeur effectue une mise à jour du dépôt Git distant.
- La mise à jour de la copie locale du référentiel est une étape manuelle.

Commande : git pull

- Lors de l'exécution de la commande, les étapes suivantes se produisent :
 - Le référentiel local (répertoire .git) est mis à jour avec le dernier commit, l'historique des fichiers, etc. à partir du dépôt Git distant.
 - Le répertoire de travail et la branche sont mis à jour avec le dernier contenu de l'étape 1.
 - Un seul commit est créé sur la branche locale avec les modifications de l'étape 1.
 - Le répertoire de travail est mis à jour avec le contenu le plus récent.

Mise à jour des référentiels (suite)

- Pour mettre à jour la copie locale du dépôt Git à partir de la branche parent, utilisez la commande suivante :

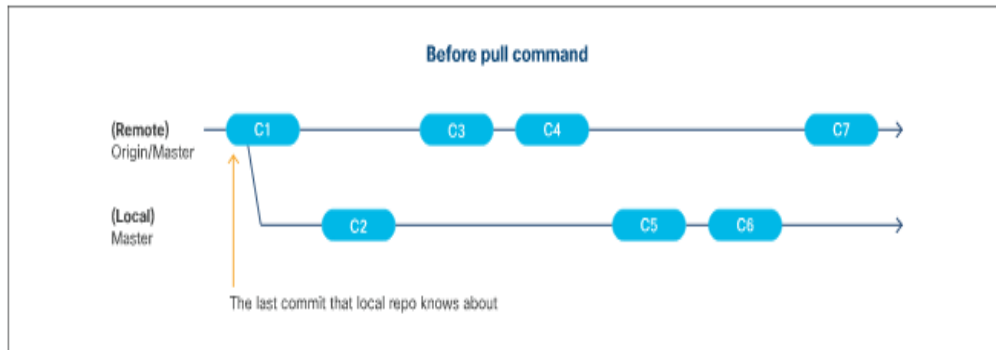
```
$ git pull
```

ou

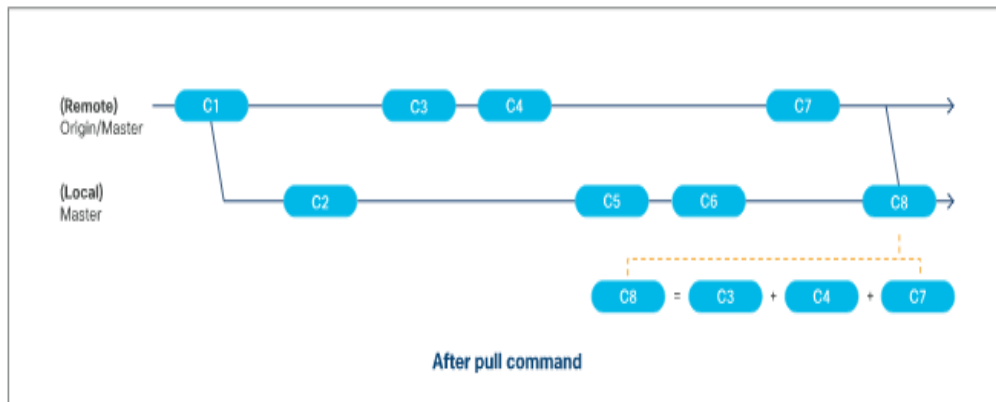
```
$ git pull origine
```

- Pour mettre à jour la copie locale du dépôt Git à partir d'une branche spécifique, utilisez la commande suivante :

```
$ git pull origine <branch>
```



```
git pull
```



Caractéristiques de branchement

Création et suppression d'une branche

Option 1 : commande git branch pour lister, créer ou supprimer une branche.

```
Branche $ git <parent branch> <branch name>
```

Option 2 : commande git checkout pour changer de branche en mettant à jour le répertoire de travail avec le contenu de la branche.

```
$ git checkout -b <parent branch> <branch name>
```

Suppression d'une branche

- Pour supprimer une branche, utilisez la commande suivante :

```
$ git branche -d <branch name>
```

Obtenir une liste de toutes les succursales

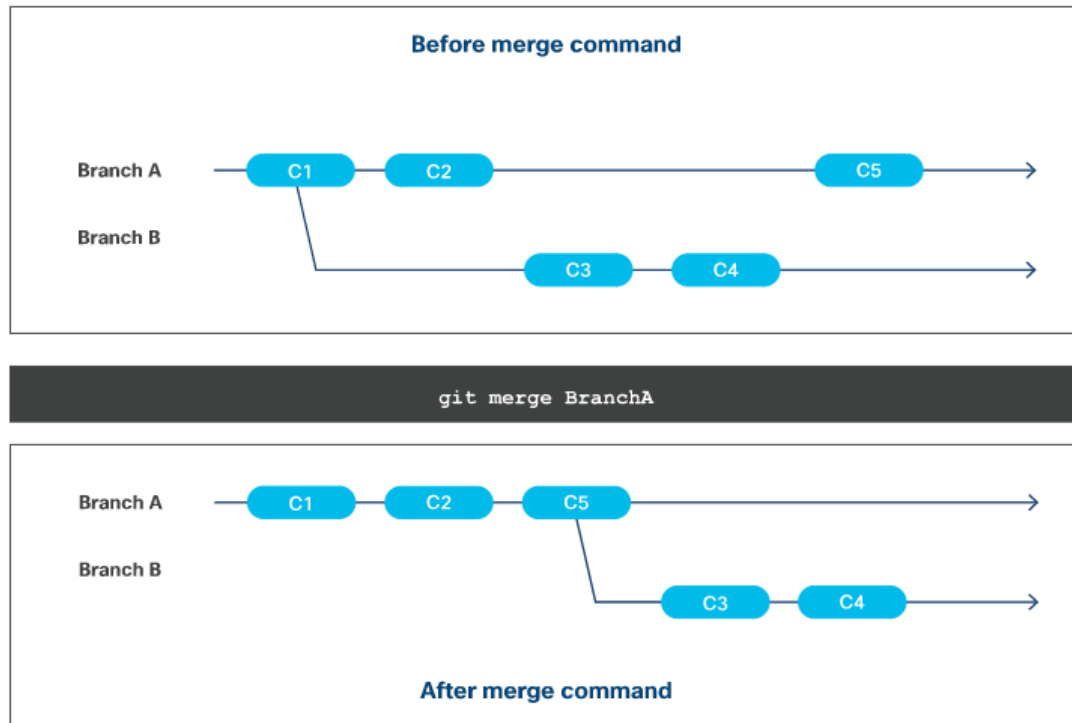
- Pour obtenir une liste de toutes les branches locales, utilisez la commande suivante :

```
$ git branche Ou $ git branche —list
```

Caractéristiques de branchement (suite)

Fusionner des branches

- Les branches divergent les unes des autres lorsqu'elles sont modifiées après leur création.
- Lorsque Git fusionne la branche, il prend les modifications/validations de la branche source et l'applique à la branche cible.
- Lors d'une fusion, seule la branche cible est modifiée.
- La branche source est intacte et reste la même.



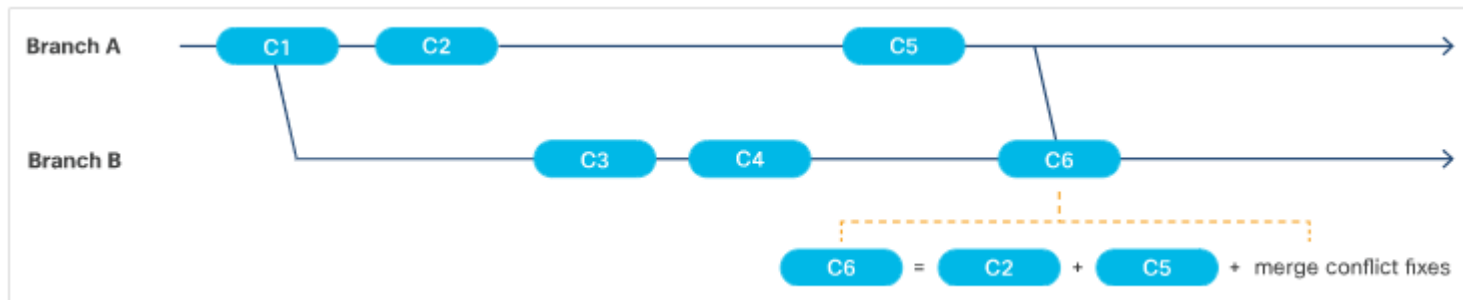
Caractéristiques de branchement (suite)

Fusion accélérée

- Une fusion rapide est lorsque l'algorithme Git est capable d'appliquer les modifications/commits de la branche source à la branche cible automatiquement et sans conflit.

Fusionner les conflits

- Un conflit de fusion est lorsque Git n'est pas en mesure d'effectuer une fusion rapide car il ne sait pas comment appliquer automatiquement les modifications des branches ensemble pour le ou les fichiers.



Caractéristiques de branchement (suite)

Exécution de la fusion

- Git fournit une commande git merge pour joindre deux ou plusieurs branches ensemble.
- **Commande** : git merge
 - Pour fusionner une branche dans la branche/référentiel actuelle du client, utilisez la commande ci-dessous :
`$ git merge <branch name>`
 - Pour fusionner une branche dans une branche qui n'est pas la branche/référentiel actuelle du client, utilisez la commande suivante :
`$ git checkout <target branch name>`
`$ git fusion <source branch name>`
 - Pour fusionner plus d'une branche dans la branche/référentiel actuelle du client, utilisez la commande ci-dessous :
`$ git merge <branch name 1>...<branch name n>`

Fichiers .diff

Qu'est-ce qu'un fichier .diff ?

- Un fichier .diff est utilisé pour montrer comment deux versions différentes d'un fichier ont changé.
- En utilisant des symboles spécifiques, ce fichier peut être lu par d'autres systèmes pour interpréter la manière dont les fichiers peuvent être mis à jour.
- Les symboles et les significations d'un fichier diff unifié sont les suivants :

Symbol	Signification
+	Indique que la ligne a été ajoutée.
-	Indique que la ligne a été supprimée.
/dev/null	Indique qu'un fichier a été ajouté ou supprimé.
ou « vierge »	Donne des lignes de contexte autour des lignes modifiées.
@@	Indicateur visuel que le prochain bloc d'informations commence. Dans les modifications d'un fichier, il peut y avoir plusieurs.
Index	Affiche les validations comparées.

Travaux pratiques - Contrôle de version des logiciels avec Git

- Au cours de ces travaux pratiques, vous aborderez les points suivants :
 - **Partie 1:** Lancer la DEVASC VM
 - **Partie 2:** Initialisation des Git
 - **Partie 3:** Mise en place et dépôt d'un fichier dans le dépôt Git
 - **Partie 4:** Gestion du dossier et suivi des modifications
 - **Partie 5:** Branches et fusions
 - **Partie 6:** Gérer les conflits de fusion
 - **Partie 7:** Intégrer Git avec GitHub

3.4 Notions de codage

Méthodes, fonctions, modules et classes

- À mesure que la taille et la complexité du projet augmentent et que d'autres développeurs (et parties prenantes) s'impliquent, des méthodes rigoureuses et des meilleures pratiques sont nécessaires pour aider les développeurs à écrire un meilleur code et à collaborer plus facilement autour de celui-ci.
- Qu'est-ce qu'un code propre ?

Code propre

- Les codes propres sont le résultat de développeurs essayant de rendre leur code facile à lire et à comprendre pour les autres développeurs.
- Ils suivent certains principes communs liés à la mise en forme, à l'organisation, à l'intuitivité des composants, à l'objectif et à la réutilisation.
- Les codes propres mettent l'accent sur la normalisation, l'organisation appropriée, la modularité, la fourniture de commentaires en ligne et d'autres caractéristiques qui aident à rendre le code auto-documenté.

Raisons pour lesquelles les développeurs veulent écrire du code propre

- Le code propre est plus facile à comprendre, plus compact et mieux organisé.
- Le code propre, étant modulaire, a tendance à être plus facile à tester en utilisant des méthodes automatisées telles que les frameworks de test unitaire.
- Le code propre, normalisé, est plus facile à scanner et à vérifier à l'aide d'outils automatisés.
- C'est tout simplement plus agréable.

Méthodes et fonctions

- Les méthodes et les fonctions sont des blocs de code qui effectuent des tâches lorsqu'elles sont exécutées.
- Voici quelques bonnes pratiques standard pour déterminer si un morceau de code doit être encapsulé (dans une méthode ou une fonction) :
 - Le code qui effectue une tâche discrète, même si elle n'est effectuée qu'une seule fois, peut être candidat à l'encapsulation.
 - Le code de tâche utilisé plus d'une fois devrait probablement être encapsulé.
- Les méthodes et les fonctions peuvent être écrites une fois et exécutées autant de fois que nécessaire.
- Si elles sont utilisées correctement, les méthodes et les fonctions simplifieront le code et réduiront le risque de bogues.
- Syntaxe d'une fonction en Python :

```
# Define the function
def functionName:
    ...blocks of code...
# Call the function
functionName()
```

Méthodes et fonctions (suite)

Arguments et paramètres

- Les arguments et les paramètres ajoutent de la flexibilité aux méthodes et aux fonctions.
- Syntaxe d'une fonction utilisant des arguments et des paramètres en Python :

```
# Define the function
def functionName(parameter1,...,parameterN):
    # You can use the parameters just like local variables
    ...blocks of code...
# Call the function
functionName("argument1", 4, {"argument3":"3"})
```

Méthodes et fonctions (suite)

Déclarations de retour

- L'instruction return fait référence à la valeur de retour spécifiée à l'aide du mot-clé return suivi d'une variable ou d'une expression. Une instruction return met fin à l'exécution d'une fonction et renvoie le contrôle à la fonction appelante.
- Lorsqu'une instruction return est exécutée, la valeur de l'instruction return est renvoyée et tout code ci-dessous est ignoré.
- Syntaxe d'une fonction avec une instruction de retour en Python :

```
# Define the function
def functionName(parameter1,...,parameterN):
    # You can use the parameters just like local variables
    ...blocks of code...
    someVariable = parameter1 * parameter2
    return someVariable
# Call the function
myVariable = functionName("argument1", 4, {"argument3":"3"})
```

Méthodes et fonctions (suite)

Méthodes par rapport aux fonctions

Méthodes	Fonctions
Les méthodes sont des blocs de code associés à un objet, généralement pour la programmation orientée objet.	Les fonctions sont des blocs de code autonomes.

Modules

- Les développeurs utilisent généralement des modules pour diviser un grand projet en petites parties afin que le code puisse être lu et compris facilement.
- Ils se composent d'un ensemble de fonctions et contiennent généralement une interface pour d'autres modules à intégrer.
- Un module est emballé sous la forme d'un seul fichier et devrait fonctionner indépendamment.
- Voici un module avec un ensemble de fonctions enregistrées dans un script appelé circleClass.py.

```
# Given a radius value, print the circumference of a circle.
# Formula for a circumference is  $c = \pi * 2 * \text{radius}$ 

class Circle:

    def __init__(self, radius):
        self.radius = radius

    def circumference(self):
        pi = 3.14
        circumferenceValue = pi * self.radius * 2
        return circumferenceValue

    def printCircumference(self):
        myCircumference = self.circumference()
        print ("Circumference of a circle with a radius of " + str(self.radius) + " is " +
str(myCircumference))
```

Classes

- Dans la plupart des langages Object-Orient Programming (OOP), et en Python, les classes sont un moyen de grouper les données et les fonctionnalités. Chaque déclaration de classe définit un nouveau type d'objet.
- Les classes peuvent avoir des variables de classe et des variables d'objet.
- De nouvelles classes peuvent être définies, en fonction des classes existantes précédemment définies, de sorte qu'elles héritent des propriétés, des membres de données et des fonctionnalités (méthodes).
- Une classe peut être instanciée (créée) plusieurs fois, et chacune avec ses propres valeurs d'attribut de données spécifiques à l'objet.

Note : Contrairement aux autres langages POO, en Python, il n'y a aucun moyen de créer des variables de classe « privées » ou des méthodes internes. Cependant, par convention, les méthodes et les variables avec un seul trait de soulignement précédent () sont considérées comme privées et ne doivent pas être utilisées ou référencées en dehors de la classe.

Travaux pratiques - Revue des classes Python

Au cours de ces travaux pratiques, vous aborderez les points suivants :

- **Partie 1:** Lancer la DEVASC VM
- **Partie 2:** Revue des fonctions, des méthodes et des classes
- **Partie 3:** Définir une fonction
- **Partie 4:** Définir une classe avec des méthodes
- **Partie 5:** Révision du script de circleClass.py

3.5 Revue et test du code

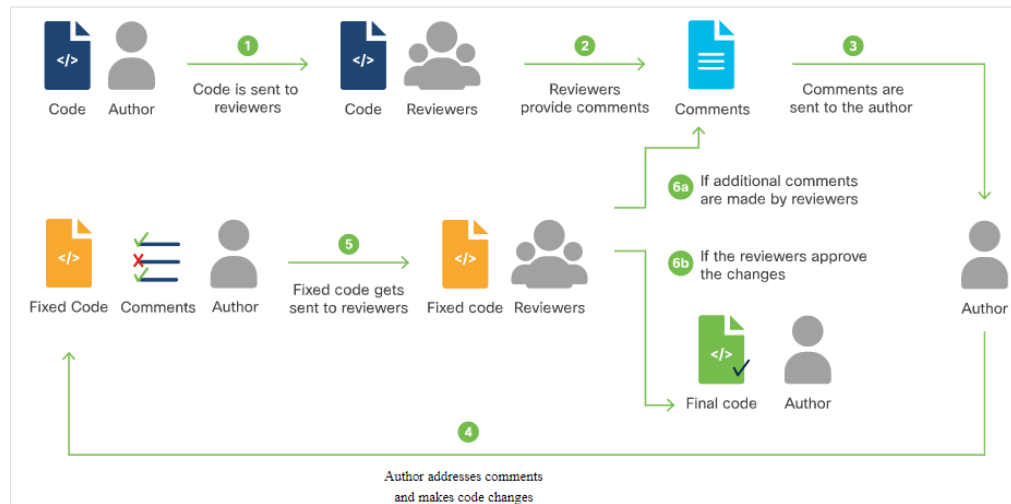
Qu'est-ce qu'un examen du code et pourquoi devriez-vous le faire ?

- Un examen de code est lorsque les développeurs regardent la base de code, un sous-ensemble de code ou des modifications de code spécifiques et fournissent des commentaires. Ces développeurs sont souvent appelés réviseurs.
- Le processus de révision du code n'intervient qu'une fois que les modifications de code sont terminées et testées.
- Le but des révisions de code est de s'assurer que le code final :
 - Est facile à lire
 - Est facile à comprendre
 - Suit les meilleures pratiques de codage
 - Utilise une mise en forme correcte
 - Est exempt de bugs
 - Dispose de commentaires et de documents appropriés
 - Est propre

Types de revues de codes

Les types de processus de revue de code les plus courants sont les suivants :

- **Revue formel du code** : Les développeurs ont une série de réunions pour examiner l'ensemble de la base de code.
- **Revue du code basé sur les modifications** : Également connu sous le nom d'examen de code assisté par un outil, cet examen examine le code qui a été modifié à la suite d'un bogue, d'une histoire d'utilisateur, d'une fonctionnalité, d'une validation, etc.
- **Revue du code à l'épaule** : Un réviseur regarde par-dessus l'épaule du développeur qui a écrit le code et fournit des commentaires.
- **Transfert de mail**: Il peut se produire à la suite des courriels automatiques envoyés par les systèmes de gestion du code source lors d'un contrôle.



Test

- Les tests de logiciels sont classiquement subdivisés en deux catégories générales :
 - **Les tests fonctionnels** visent à déterminer si le logiciel fonctionne correctement. Est-ce qu'il se comporte comme prévu dans un sens logique, depuis les niveaux de détail les plus bas examinés avec les tests unitaires, jusqu'aux niveaux plus élevés de complexité explorés dans les tests d'intégration ?
 - **Les tests non fonctionnels** examinent la facilité d'utilisation, les performances, la sécurité, la résilience, la conformité, la localisation et bien d'autres problèmes. Ce type de test permet de déterminer si le logiciel est adapté à ses besoins, fournit la valeur voulue et minimise les risques.
- Les développeurs capturent les exigences de conception sous forme de tests, puis écrivent des logiciels pour réussir ces tests. C'est ce qu'on appelle Test-Driven Development (TDD).

Tests unitaires

- Test fonctionnel détaillé de petits morceaux de code (lignes, blocs, fonctions, classes et autres composants isolés) est appelé Unit Testing.
- Ces frameworks de test sont des logiciels qui vous permettent de faire des assertions sur des conditions testables et de déterminer si ces assertions sont valides à un moment de l'exécution.
- Exemples de frameworks de test pour Python :

PyTest

- PyTest est pratique. Il exécute automatiquement tous les scripts qui commencent par `test_` ou se terminent par `_test.py` et dans ces scripts, exécute automatiquement toutes les fonctions commençant par `'test_'` ou `'tests_'`.
- Nous pouvons tester unitaire un morceau de code en le copiant dans un fichier, en important `pytest`, en ajoutant des fonctions de test nommées, en enregistrant le fichier sous un nom de fichier qui commence également par `'tests_'`, et en l'exécutant avec PyTest.

unittest

- Le framework `unittest` exige une syntaxe différente de PyTest.
- Pour `unittest`, vous devez sous-classer la classe `TestCase` intégrée et tester en remplaçant ses méthodes intégrées ou en ajoutant de nouvelles méthodes dont les noms commencent par `'test_'`.

Revue et test du code

Test d'intégration

- Les tests d'intégration garantissent que toutes les unités individuelles s'intègrent correctement pour faire une application complète.
- L'exécution du code avec PyTest produit une sortie comme indiqué dans l'image :

Remarque : Vous pouvez exécuter ce script sur votre machine virtuelle en utilisant pytest. Cependant, la compréhension de la sortie et la correction des erreurs dépasse la portée de ce cours.

```
===== test session starts =====
platform linux2 -- Python 2.7.15+, pytest-3.3.2, py-1.5.2, pluggy-0.6.0
rootdir: /home/ubuntu/deploy/sample, inifile:
collected 1 item
test_sample_app.py F [100%]
===== FAILURES =====
_____ test_setconfig _____

    def test_setconfig():
        setUp()
        set_config("TESTVAL")
>       assert get_config() == "ESTVAL"
E       AssertionError: assert 'TESTVAL' == 'ESTVAL'
E       - TESTVAL
E       ? -
E       + ESTVAL
test_sample_app.py:21: AssertionError
----- Captured log call -----
connectionpool.py      225 DEBUG    Starting new HTTP connection (1): localhost:80
connectionpool.py      437 DEBUG    http://localhost:80 "GET /get_config HTTP/1.1" 200 7
connectionpool.py      225 DEBUG    Starting new HTTP connection (1): localhost:80
connectionpool.py      437 DEBUG    http://localhost:80 "GET /config_action?dbhost=TESTVAL HTTP/1.1"
200 30
connectionpool.py      225 DEBUG    Starting new HTTP connection (1): localhost:80
connectionpool.py      437 DEBUG    http://localhost:80 "GET /get_config HTTP/1.1" 200 7
===== 1 failed in 0.09 seconds =====
```

Développement piloté par les tests (TDD)

- Si vous voulez tester pour valider la conception de l'application à la lumière des exigences, implique que vous devez écrire le code de test avant d'écrire le code de l'application.
- Après avoir exprimé les exigences dans votre code de test, vous pouvez ensuite écrire le code d'application jusqu'à ce qu'il réussisse les tests que vous avez créés dans le code de test.
- Le modèle de base de TDD est un processus répété en cinq étapes :
 - Créez un nouveau test.
 - Exécutez des tests pour voir si un échoue pour des raisons inattendues.
 - Écrivez le code de l'application pour réussir le nouveau test.
 - Exécutez des tests pour voir s'il y a un échec.
 - Refactoriser et améliorer le code de l'application.

Travaux pratiques - Créer un test unitaire Python

Au cours de ces travaux pratiques, vous aborderez les points suivants :

- **Partie 1:** Lancer la DEVASC VM
- **Partie 2:** Explorer les options dans le cadre du test unitaire
- **Partie 3:** Tester une fonction python avec unittest

3.6 Comprendre les formats de données

Formats de données

- Les API de repos vous permettent d'échanger des informations avec des services et équipements distants.
- Les trois formats standard les plus populaires pour l'échange d'informations avec les API distantes sont XML, JSON et YAML.
- L'analyse XML, JSON ou YAML est une exigence fréquente d'interaction avec les API. Un modèle souvent rencontré dans les implémentations d'API REST est le suivant :
 - Authentifier, généralement en affichant une combinaison utilisateur/mot de passe et en récupérant un jeton expirant pour l'authentification des demandes ultérieures.
 - Exécutez une requête GET vers un point de terminaison donné (authentification si nécessaire) pour récupérer l'état d'une ressource, demandant XML, JSON ou YAML comme format de sortie.
 - Modifiez le fichier XML, JSON ou YAML renvoyé.
 - Exécutez un POST (ou PUT) sur le même point de terminaison (encore une fois, en s'authentifiant si nécessaire) pour modifier l'état de la ressource, en demandant à nouveau XML, JSON ou YAML comme format de sortie et en l'interprétant selon les besoins pour déterminer si l'opération a réussi.

XML

- XML (Extensible Markup Language) est une méthodologie générique pour envelopper des données textuelles dans des balises symétriques pour indiquer la sémantique.
- Il est un dérivé de Structured, Generalized Markup Language (SGML), et également le parent de HyperText Markup Language (HTML). Les noms de fichiers XML se terminent généralement par « .xml ».

Exemple de document XML

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Instance list -->
<vms>
  <vm>
    <vmid>0101af9811012</vmid>
    <type>t1.nano</type>
  </vm>
  <vm>
    <vmid>0102bg8908023</vmid>
    <type>t1.micro</type>
  </vm>
</vms>
```

XML (suite)

- **Corps du document XML** : À l'exception des deux premières lignes d'un document XML, le reste du document est considéré comme le corps.
- **Noms de balises définis par l'utilisateur** : les noms de balises XML sont définis par l'utilisateur. Si vous composez du XML pour votre propre application, choisissez des noms de balises qui expriment clairement la signification des éléments de données, leurs relations et leur hiérarchie.
- **Encodage de caractères spécial** : Les données sont transmises en XML sous forme de texte lisible.
- **Prologue XML** : Le prologue XML est la première ligne d'un fichier XML.
- **Commentaires en XML** : les fichiers XML peuvent inclure des commentaires, en utilisant la même convention de commentaire utilisée dans les documents HTML.
- **Attributs XML** : XML vous permet d'incorporer des attributs dans des balises pour transmettre des informations supplémentaires.

XML (suite)

- **les espaces de noms XML**

- Les espaces de noms sont définis par l'IETF et d'autres autorités Internet, organisations et autres entités, et leurs schémas sont généralement hébergés en tant que documents publics sur le Web.
- Les espaces de noms sont identifiés par des URI (Uniform Resource Names) pour rendre les documents persistants accessibles sans que le demandeur ait à se préoccuper de leur emplacement.
- L'exemple de code ci-dessous montre l'utilisation d'un espace de noms, défini comme la valeur d'un attribut xmlns, pour affirmer que le contenu d'un appel de procédure distante XML doit être interprété conformément à la norme NETCONF 1.0 héritée.

```
<rpc message-id="101" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">  
  <kill-session>  
    <session-id>4</session-id>  
  </kill-session>  
</rpc>
```

XML (suite)

- **Interprétation du XML**

- Dans l'exemple des espaces de noms XML, la structure est représentée sous la forme d'une liste ou d'un tableau unidimensionnel (appelé 'instances') d'objets (chacun identifié comme 'instance' par des balises de bracketing). Chaque objet d'instance contient deux paires clé-valeur indiquant un ID d'instance unique et un type de serveur VM.
- Une structure de données Python sémantique équivalente peut être déclarée comme indiqué ci-dessous :

```
vms [  
  {  
    {"vmid": "0101af9811012"},  
    {"type": "t1.nano"}  
  },  
  {  
    {"vmid": "0102bg8908023"},  
    {"type": "t1.micro"}  
  }  
]
```

JSON

- JSON, ou JavaScript Object Notation, est un format de données dérivé de la façon dont les littéraux d'objets complexes sont écrits en JavaScript.
- Les noms de fichiers JSON se terminent généralement par « .json ».
- Voici un exemple de fichier JSON, contenant deux valeurs qui sont des chaînes de texte, l'une est une valeur booléenne et deux sont des tableaux :

```
{
  "edit-config":
  {
    "default-operation": "merge",
    "test-operation": "set",
    "some-integers": [2,3,5,7,9],
    "a-boolean": true,
    "more-numbers": [2.25E+2,-1.0735],
  }
}
```

JSON (suite)

- **Types de données de base JSON** : les types de données de base JSON incluent des nombres, des chaînes, des booléens ou des nulls.
- **Objets JSON** : Comme en JavaScript, les objets individuels dans JSON comprennent des paires clé/valeur, qui peuvent être entourées d'accolades, individuellement.
- **Cartes et listes JSON**: Dans ce cas, chaque paire clé/valeur n'a pas besoin de son propre ensemble de crochets, mais l'objet entier le fait. Les objets composés JSON peuvent être profondément imbriqués, avec une structure complexe. Il peut également exprimer des tableaux ordonnés JavaScript (ou « listes ») de données ou d'objets.
- **Aucun commentaire dans JSON** : Contrairement à XML et YAML, JSON ne prend en charge aucun type de méthode standard pour inclure des commentaires non analysés dans le code.
- **Espaces insignifiants** : les espaces dans JSON ne sont pas significatifs, et les fichiers peuvent être indentés en utilisant des onglets ou des espaces comme vous le souhaitez, ou pas du tout.

YAML

- YAML Ain't Markup Language (YAML) est un sur-ensemble de JSON conçu pour une lisibilité humaine encore plus facile.
- En tant que sur-ensemble de JSON, les analyseurs YAML peuvent généralement analyser des documents JSON (mais pas vice-versa).
- Par conséquent, YAML est meilleur que JSON à certaines tâches, y compris la possibilité d'intégrer JSON directement (y compris les guillemets) dans les fichiers YAML.

```
---
edit-config:
  a-boolean: true
  default-operation: merge
  more-numbers:
    - 225.0
    - -1.0735
  some-integers:
    - 2
    - 3
    - 5
    - 7
    - 9
  test-operation: set
---
```

YAML (suite)

- **Structure de fichiers** YAML : Les fichiers YAML s'ouvrent conventionnellement avec trois tirets (— seuls sur une ligne) et se terminent par trois points (... de même).
- **Types de données** YAML : Les types de données de base YAML incluent des nombres, des chaînes, des booléens ou des nulls.
- **Objets de base** : Dans YAML, les types de données de base sont assimilés aux clés.
- **YAML Indentation et structure de fichier** : YAML indique sa hiérarchie à l'aide de l'indentation.
- **Cartes et listes** : YAML représente facilement des types de données plus complexes, tels que des cartes contenant plusieurs paires clé/valeur et des listes ordonnées.
 - Les cartes sont généralement exprimées sur plusieurs lignes, en commençant par une clé d'étiquette et un deux-points, suivis par des membres, en retrait sur les lignes suivantes :

```
mymap:  
  myfirstkey: 5  
  mysecondkey: The quick brown fox
```

YAML (suite)

- Les listes (tableaux) sont représentées par des membres en retrait optionnel précédés d'un tiret et d'un espace uniques :

```
mylist:  
  - 1  
  - 2  
  - 3
```

- Les cartes et les listes peuvent également être représentées dans une soi-disant « syntaxe de flux », qui ressemble beaucoup à JavaScript ou Python :

```
mymap: { myfirstkey: 5, mysecondkey: The quick brown fox}  
mylist: [1, 2, 3]
```

YAML (suite)

- **Longues chaînes** :Elles sont représentées à l'aide d'une syntaxe « pliante », où les sauts de ligne sont présumés être remplacés par des espaces lorsque le fichier est analysé/consommé, ou dans une syntaxe sans pliage.

```
mylongstring: >
  This is my long string
  which will end up with no linebreaks in it
myotherlongstring: |
  This is my other long string
  which will end up with linebreaks as in the original
```

YAML (suite)

- **Commentaires** : Les commentaires dans YAML peuvent être insérés n'importe où sauf dans une longue chaîne littérale, et sont précédés du signe de hachage et d'un espace.

```
# this is a comment
```

- **Plus de fonctionnalités YAML** : YAML a beaucoup plus de fonctionnalités, le plus souvent rencontrées lors de l'utilisation dans le contexte de langages spécifiques, comme Python, ou lors de la conversion en JSON ou d'autres formats. Par exemple, YAML 1.2 prend en charge les schémas et les balises, qui peuvent être utilisés pour désambiguer l'interprétation des valeurs.

Par exemple, pour forcer un nombre à être interprété comme une chaîne, vous pouvez utiliser le `!str`, qui fait partie du schéma YAML « Failsafe » :

```
mynumericstring: !!str 0.1415
```

Analyse et sérialisation

- Analyser signifie analyser un message, le diviser en ses parties constitutives, et comprendre leurs objectifs en contexte.
- La sérialisation est à peu près le contraire de l'analyse.
- Les langages de programmation populaires tels que Python intègrent généralement des fonctions d'analyse faciles à utiliser qui peuvent accepter les données renvoyées par une fonction d'E/S et produire une structure de données interne sémantique équivalente contenant des données typées valides.
- Du côté sortant, ils contiennent des sérialiseurs qui transforment les structures de données internes en messages sémantiques équivalents formatés sous forme de chaînes de caractères.

Travaux pratiques - Analyser différents types de données avec Python

Au cours de ces travaux pratiques, vous aborderez les points suivants :

- **Partie 1:** Lancer la DEVASC VM
- **Partie 2:** Analyser XML en Python
- **Partie 3:** Parse JSON en Python
- **Partie 4:** Parse YAML en python

3.7 Résumé du développement et de la conception de logiciels

Qu'ai-je appris dans ce module ?

- Six phases de SDLC : Exigences et analyse, conception, implémentation, test, déploiement et maintenance.
- Trois modèles de développement logiciel populaires sont Waterfall, Agile et Lean.
- Le modèle de conception MVC simplifie le développement d'applications qui dépendent des interfaces utilisateur graphiques.
- Le contrôle de version conserve l'historique des modifications apportées à un fichier. Types de systèmes de contrôle de version : Local, Centralisé et Distribué.
- Git est une implémentation open source d'un système de contrôle de version distribué et dispose de deux types de référentiels : local et distant.
- Le code propre est le résultat de développeurs essayant de rendre leur code facile à lire et à comprendre pour les autres développeurs.
- La révision du code implique l'examen d'une base de code, d'un sous-ensemble de code ou d'un changement de code spécifique pour fournir des commentaires.
- Trois formats standard les plus populaires pour l'échange d'informations avec des API distantes : XML, JSON et YAML.
- L'analyse nécessite d'analyser un message, de le diviser en parties constitutives et de comprendre leurs objectifs en contexte. La sérialisation est à peu près le contraire.

Nouveaux termes et commandes

- | | | |
|--|--|---|
| <ul style="list-style-type: none">• Software Development Life Cycle (SDLC)• User experience (UX)• Software Requirement Specification (SRS)• Agile Scrum• Lean• Extreme Programming (XP)• Feature-Driven Development (FDD)• Sprints• Backlog• User stories• Scrum Teams• Model-View-Controller (MVC) | <ul style="list-style-type: none">• Centralized Version Control Systems (CVCS)• Distributed Version Control System (DVCS)• Git• Branching• GitHub• Arguments• Parameters• Object-Orient Programming (OOP)• Formal Code Review• Change-Based Code Review• Over-the-Shoulder Code Review | <ul style="list-style-type: none">• Test-Driven Development (TDD)• Unit Testing• Software Development Kits (SDKs)• XML• JSON• YAML• Application Programming Interfaces (APIs)• REpresentational State Transfer (REST)• Long Strings• Parsing• Serializing |
|--|--|---|

